Red Hat OpenShift AI Self-Managed 2.25

Working with machine learning features

Store, manage, and serve features to machine learning models with Feature Store.

Last Updated: 2025-10-31

Red Hat OpenShift AI Self-Managed 2.25 Working with machine learning features

Store, manage, and serve features to machine learning models with Feature Store.

Legal Notice

Abstract

Feature Store provides a framework for storing, managing, and serving features to machine learning models by using your existing infrastructure and data stores.

Table of Contents

PREFACE 3

CHAPTER 1. OVERVIEW OF MACHINE LEARNING FEATURES AND FEATURE STORE 4
1.1. AUDIENCE FOR FEATURE STORE 4
1.2. OVERVIEW OF MACHINE LEARNING FEATURES 5
1.3. OVERVIEW OF FEATURE STORE 5
1.4. FEATURE STORE WORKFLOW 7
1.5. SETTING UP THE FEATURE STORE USER INTERFACE FOR INITIAL USE 8
1.6. ADDITIONAL RESOURCES 9

CHAPTER 2. CONFIGURING FEATURE STORE 10
2.1. SETTING UP FEATURE STORE 10
2.1.1. Before you begin 10
2.1.2. Enabling the Feature Store component 12
2.1.3. Creating a Feature Store instance in a data science project 13
2.1.4. Adding feature definitions and initializing your Feature Store instance 16
2.1.4.1. Specifying files to ignore 17
2.1.5. Viewing Feature Store objects in the web-based UI 18
2.2. CUSTOMIZING YOUR FEATURE STORE CONFIGURATION 20
2.2.1. Configuring an offline store 20
2.2.2. Configuring an online store 22
2.2.3. Configuring the feature registry 23
2.2.4. Example PVC configuration 24
2.2.5. Editing an existing Feature Store instance 25

CHAPTER 3. DEFINING MACHINE LEARNING FEATURES 27
3.1. SETTING UP YOUR WORKING ENVIRONMENT 27
3.2. ABOUT FEATURE DEFINITIONS 28
3.3. SPECIFYING THE DATA SOURCE FOR FEATURES 28
3.4. ABOUT ORGANIZING FEATURES BY USING ENTITIES 29
3.5. CREATING FEATURE VIEWS 30

CHAPTER 4. RETRIEVING FEATURES FOR MODEL TRAINING 33
4.1. MAKING FEATURES AVAILABLE FOR REAL-TIME INFERENCE 33
4.2. RETRIEVING ONLINE FEATURES FOR MODEL INFERENCE 34

PREFACE

Feature Store provides an interface between machine learning models and data.

CHAPTER 1. OVERVIEW OF MACHINE LEARNING FEATURES AND FEATURE STORE

IMPORTANT

Feature Store is currently available in Red Hat OpenShift AI 2.25 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

A machine learning (ML) feature is a measurable property or attribute within a dataset that a machine learning model can analyze to learn patterns and make decisions. Examples of features include a customer's purchase history, demographic data like age and location, weather conditions, and financial market data. You can use these features to train models for tasks such as personalized product recommendations, fraud detection, and predictive maintenance.

Feature Store is a Red Hat OpenShift AI component that provides a centralized repository that stores, manages, and serves machine learning features for both training and inference purposes.

1.1. AUDIENCE FOR FEATURE STORE

The target audience for Feature Store is ML platform and MLOps teams with DevOps experience in deploying real-time models to production. Feature Store also helps these teams build a feature platform that improves collaboration between data engineers, software engineers, machine learning engineers, and data scientists.

For Data Scientists

Feature Store is a tool where you can define, store, and retrieve your features for both model development and model deployment. By using Feature Store, you can focus on what you do best: build features that power your AI/ML models and maximize the value of your data.

For MLOps Engineers

Feature Store is a library that connects your existing infrastructure, such as online database, application server, microservice, analytical database, and orchestration tooling. By using Feature Store, you can focus on maintaining a resilient system, instead of implementing features for data scientists.

For Data Engineers

Feature Store provides a centralized catalog for storing feature definitions, allowing you to maintain a single source of truth for feature data. It provides the abstraction for reading and writing to many different types of offline and online data stores. Using the provided Python SDK or the feature server service, you can write data to the online and offline stores and then read out that data in either batch scenarios for model training or low-latency online scenarios for model inference.

For AI Engineers

Feature Store provides a platform designed to scale your AI applications by enabling seamless integration of richer data and facilitating fine-tuning. With Feature Store, you can optimize the performance of your AI models while ensuring a scalable and efficient data pipeline.

1.2. OVERVIEW OF MACHINE LEARNING FEATURES

In machine learning, a *feature*, also referred to as a field, is an individual measurable property. A feature is used as an input signal to a predictive model. For example, if a bank's loan department is trying to predict whether an applicant should be approved for a loan, a useful feature might be whether they have filed for bankruptcy in the past or how much credit card debt they currently carry.

Table 1.1. A feature represents a column in a data table

customer_id	avg_cc_balance	credit_score	bankruptcy
1005	500.00	730	0
982	20000.00	570	2
1001	1400.00	600	0

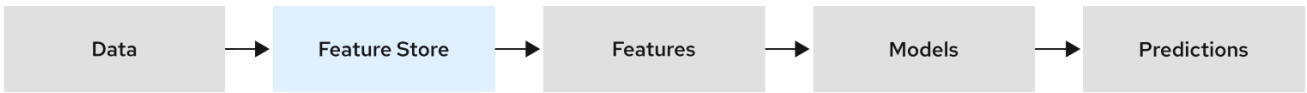
Features are prepared data that help machine learning models understand patterns in the world. Feature engineering is the process of selecting, manipulating, and transforming raw data into features that can be used in supervised learning. As shown in the table, a feature refers to an entire column in a dataset, for example, **credit_score**. A feature value refers to a single value in a feature column, such as **730**.

1.3. OVERVIEW OF FEATURE STORE

Feature Store is an OpenShift AI component that provides an interface between models and data. It is based on the Feast open source project. Feature Store provides a framework for storing, managing, and serving features to machine learning models by using your existing infrastructure and data stores. It facilitates the retrieval of feature data from different data sources to generate and manage features by providing unified feature management capabilities.

The following figure shows where Feature Store fits in the ML workflow. In an ML workflow, features are inputs to ML models. The ML workflow starts with many types of relevant data, such as transactional data, customer references, and product data. The data comes from a variety of databases and data sources. From this data, ML engineers use Feature Store to curate features. The features are input to models and the models can then use the data from the features to make predictions.

Figure 1.1. Feature Store in the ML workflow



Feature Store is a machine learning data system that provides the following capabilities:

- Runs data pipelines that transform raw data into feature values
- Stores and manages feature data
- Serves feature data consistently for training and inference purposes

- Manages features consistently across offline and online environments
- Powers one model or thousands simultaneously with fresh, reusable features, on demand

Feature Store is a centralized hub for storing, processing, and accessing commonly-used features that enables users in your ML organization to collaborate. When you register a feature in a Feature Store, it becomes available for immediate reuse by other models across your organization. The Feature Store registry reduces duplication of data engineering efforts and allows new ML projects to bootstrap with a library of curated, production-ready features.

Feature Store provides consistency in model training and inference, promotes collaboration and usability across multiple projects, monitors lineage and versioning of models for data drifts, leaks, and training skews, and seamlessly integrates with other MLOps tools. Feature Store remotely manages data stored in other systems, such as BigQuery, Snowflake, DynamoDB, and Redis, to make features consistently available at training / serving time.

Feature Store performs the following tasks:

- Stores features in offline and online stores
- Registers features in the registry for sharing
- Serves features to ML models

ML platform teams use Feature Store to store and serve features consistently for offline training, such as batch-scoring, and online real-time model inference.

Feature Store consists of the following key components:

Registry

A central catalog of all feature definitions and their related metadata. It allows ML engineers and data scientists to search, discover, and collaborate on new features. The registry exposes methods to apply, list, retrieve, and delete features.

Offline Store

The data store that contains historical data for scale-out batch scoring or model training. The offline store persists batch data that has been ingested into Feature Store. This data is used for producing training datasets. Examples of offline stores include Dask, Snowflake, BigQuery, Redshift, and DuckDB.

Online Store

The data store that is used for low-latency feature retrieval. The online store is used for real-time inference. Examples of online stores include Redis, GCP Datastore, and DynamoDB.

Server

A feature server that serves pre-computed features online. There are three Feature Store servers:

- **The online feature server** - A Python feature server that is an HTTP endpoint that serves features with JSON I/O. You can write and read features from the online store using any programming language that can make HTTP requests.
- **The offline feature server** - An Apache Arrow Flight Server that uses the gRPC communication protocol to exchange data. This server wraps calls to existing offline store implementations and exposes interfaces as Arrow Flight endpoints.
- **The registry server** - A server that uses the gRPC communication protocol to exchange data. You can communicate with the server using any programming language that can make gRPC requests.

UI

A web-based graphical user interface (UI) for viewing all the Feature Store objects and their relationships with each other.

Feature Store provides the following software capabilities:

- A Python SDK for programmatically defining features and data sources
- A Python SDK for reading and writing features to offline and online data stores
- An optional feature server for reading and writing features (useful for non-python languages) by using APIs
- A web-based UI for viewing and exploring information about features defined in the project
- A command line interface (CLI) for viewing and updating feature information

1.4. FEATURE STORE WORKFLOW

The Feature Store workflow involves the following tasks OpenShift cluster administrators, and machine learning (ML) engineers or data scientists:

Note: This Feature Store workflow describes a local implementation that is available in this Technology Preview release.

Cluster administrator

Installs and configures Feature Store, as described in *Chapter 2. Configuring Feature Store* :

1. Installs OpenShift AI.
2. Enables the Feature Store component by using the Feature Store operator.
3. Creates a data science project.
4. In the data science project, creates a Feature Store instance by using a **feast.yaml** file that specifies the offline and online stores.
5. Sets up Feature Store so that ML Engineers and data scientists can push and retrieve features to use for model training and inference.

ML Engineer or data scientist

- Prepares features, as described in *Chapter 3: Defining features* :
    1. Creates a feature definition file.
    2. Defines the data sources and other Feature Store objects.
    3. Makes features available for real-time inference.
- Prepares features for model training and real-time inference, as described in *Chapter 4. Retrieving features for model training*:
    1. Makes features available to models.
    2. Uses **feast** Python APIs to retrieve features for model training and inference.

1.5. SETTING UP THE FEATURE STORE USER INTERFACE FOR INITIAL USE

You can use a web-based user interface to simplify and accelerate the creation of model development features. This visual interface helps you explore and understand your Feature Store.

You can use the Feature Store UI to access a centralized catalog of features and metadata, such as transformation logic and materialization job status. You can also view features, manage entities, and use lineage and search capabilities.

You must enable the Feature Store UI before you can use it.

Prerequisites

- You have Administrator access.
- You have enabled the Feast Operator.
- You have created a Feature Store CRD, as described in *Creating a Feature Store instance in a data science project*.
- Your REST API server is running.

Procedure

1. Log in to the OpenShift AI Dashboard and click the **Feature Store** tab on the left navigation.
2. On the **Overview** page, click the **create FeatureStore** button.
3. Add YAML definitions to enable the user interface.
4. Edit the following label to enable the UI:
    a. **Label:**
    b. **feature-store-ui: enabled**
    c. This creates a pod and initiates the service registry.
5. Click the options icon (⋮) and choose **Start job**.
6. Click on the **Jobs** tab and then **Logs** on the left navigation, to confirm that the CronJob is running.
7. Navigate to the **Feature views** tab on the left navigation, and you will be able to see your new UI.

Verification

Click **Overview**. If your Feature Store user interface was created, you see the following cards: * Entities * Data sources * Datasets * Features * Feature views * Feature services

Additional resources

Configuring the feature registry

1.6. ADDITIONAL RESOURCES

- For example Feature Store CRD configurations, see the Feast Operator configuration samples.
- For details about the Feast CRD APIs, see the Feast API documentation.
- For information on how to implement machine learning features, see the Feast documentation.
- For end-to-end use case examples of how Feature Store can benefit your AI/ML workflows, see Feast Getting Started: Use Cases .

CHAPTER 2. CONFIGURING FEATURE STORE

As a cluster administrator, you can install and manage Feature Store as a component in the Red Hat OpenShift AI Operator configuration.

2.1. SETTING UP FEATURE STORE

As a cluster administrator, you must complete the following tasks to set up Feature Store:

1. Enable the Feature Store component.
2. Create a data science project and add a Feature Store instance.
3. Initialize the Feature Store instance.
4. Set up Feature Store so that ML Engineers and data scientists can push and retrieve features to use for model training and inference.

2.1.1. Before you begin

Before you implement Feature Store in your machine learning workflow, you must have the following information:

Knowledge of your data and use case

You must know your use case and your raw underlying data so that you can identify the properties or attributes that you want to define as features. For example, if you are developing machine learning (ML) models that detect possible credit card fraud transactions, you would identify data such as purchase history, transaction location, transaction frequency, or credit limit.
With Feature Store, you define each of those attributes as a feature. You group features that share a conceptual link or relationship together to define an entity. You define entities to map to the domain of your use case. Not all features must be in an entity.

Knowledge of your data source

You must know the source of the raw data that you want to use in your ML workflow. When you configure the Feature Store online and offline stores and the feature registry, you must specify an environment that is compatible with the data source. Also, when you define features, you must specify the data source for the features.
Feature Store uses a time-series data model to represent data. This data model is used to interpret feature data in data sources in order to build training datasets or materialize features into an online store.

You can connect to the following types of data sources:

Batch data source

A method of collecting and processing data in discrete chunks or batches, rather than continuously streaming it. This approach is commonly used for large datasets or when real-time processing is not essential. In a data processing context, a batch data source defines the connection to the data-at-rest source, allowing you to access and process data in batches. Examples of batch data sources include data warehouses (for example, BigQuery, Snowflake, and Redshift) or data lakes (for example, S3 and GCS). Typically, you define a batch data source when you configure the Feature Store offline store.

Stream data source

The origin of data that is continuously flowing or emitted for online, real-time processing. Feature Store does not have native streaming integrations, but it facilitates push sources that allow you to push features into Feature Store. You can use Feature Store for training or batch scoring (offline), for real-time feature serving (online), or for both. Typically, you define a stream data source when you configure the Feature Store online store.

You can use the following data sources with Feature Store:

Data sources for online stores

- SQLite
- Snowflake
- Redis
- Dragonfly
- IKV
- Datastore
- DynamoDB
- Bigtable
- PostgreSQL
- Cassandra + Astra DB
- Couchbase
- MySQL
- Hazelcast
- ScyllaDB
- Remote
- SingleStore

For details on how to configure these online stores, see the Feast reference documentation for online stores.

Data sources for offline stores

- Dask
- Snowflake
- BigQuery
- Redshift
- DuckDB

An offline store is an interface for working with historical time-series feature values that are stored in data sources. Each offline store implementation is designed to work only with the corresponding data source.

Offline stores are useful for the following purposes:

- To build training datasets from time-series features.
- To materialize (load) features into an online store to serve those features at low-latency in a production setting.

You can use only a single offline store at a time. Offline stores are not compatible with all data sources; for example, the BigQuery offline store cannot be used to query a file-based data source.

For details on how to configure these offline stores, see the Feast reference documentation for offline stores.

Data sources for the feature registry

- Local
- S3
- GCS
- SQL
- Snowflake

For details on how to configure these registry options, see the Feast reference documentation for the registry.

2.1.2. Enabling the Feature Store component

To allow the ML engineers and data scientists in your organization to work with machine learning features, you must enable the Feature Store component in Red Hat OpenShift AI.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed OpenShift AI.

Procedure

1. In the OpenShift console, click **Operators → Installed Operators**.
2. Click the **Red Hat OpenShift AI** Operator.
3. Click the **Data Science Cluster** tab.
4. Click the default instance name (for example, **default-dsc**) to open the instance details page.
5. Click the **YAML** tab.
6. Edit the **spec:components** section. For the **feastoperator** component, set the **managementState** field to **Managed**:

```
spec:
  components:
    feastoperator:
      managementState: Managed
```

7. Click **Save**.

Verification

Check the status of the **feast-operator-controller-manager-<pod-id>** pod:

1. Click **Workloads → Deployments**.
2. From the **Project** list, select **redhat-ods-applications**.
3. Search for the **feast-operator-controller-manager** deployment.
4. Click the **feast-operator-controller-manager** deployment name to open the deployment details page.
5. Click the **Pods** tab.
6. View the pod status.

When the status of the **feast-operator-controller-manager-<pod-id>** pod is **Running**, Feature Store is enabled.

Next Step

- Create a Feature Store instance in a data science project.

2.1.3. Creating a Feature Store instance in a data science project

You can add an instance of Feature Store to a data science project by creating a custom resource definition (CRD) in the OpenShift console.

The following example shows the minimum requirements for a Feature Store CR YAML file:

```
apiVersion: feast.dev/v1alpha1
kind: FeatureStore
metadata:
  name: sample
spec:
  feastProject: my_feast_project
```

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have enabled the Feature Store component, as described in *Enabling the Feature Store component*.
- You have set up your database infrastructure for the online store, offline store, and registry.

For an example of setting up and running PostgreSQL (for the registry) and Redis (for the online store), see the Feature Store Operator quick start example: https://github.com/feast-dev/feast/tree/stable/examples/operator-quickstart.

- You have created a data science project, as described in Creating a data science project . In the following procedure, **my-ds-project** is the name of the data science project.

Procedure

1. In the OpenShift console, click the **Quick Create** () icon and then click the **Import YAML** option.
2. Verify that your data science project is the selected project.
3. Copy the following code and paste it into the YAML editor:

```
apiVersion: feast.dev/v1alpha1
kind: FeatureStore
metadata:
  name: sample-git
spec:
  feastProject: credit_scoring_local
  feastProjectDir:
    git:
      url: https://github.com/feast-dev/feast-credit-score-local-tutorial
      ref: 598a270
```

   The **spec.feastProjectDir** references a Feature Store project that is in the Git repository for a Credit Store tutorial.
4. Optionally, change the **metadata.name** for the Feature Store instance.
5. Optionally, edit **feastProject**, which is the namespace for organizing your Feature Store instance. Note that this project is not the OpenShift AI data science project.
6. Click **Create**.

When you create the Feature Store CR in OpenShift, Feature Store starts a remote online feature server, and configures a default registry and an offline store with the local provider.

A *provider* is a customizable interface that provides default Feature Store components, such as the registry, offline store, and online store, that target a specific environment, ensuring that these components can work together seamlessly. The local provider uses the following default settings:

- **Registry:** A SQL registry or local file
- **Offline store:** A Parquet file
- **Online store:** SQLite

Verification

1. In the OpenShift console, select **Workloads → Pods**.
2. Make sure that your data science project (for example, my-ds-project) is selected.

3. Find the pod that has the **feast-** prefix, followed by the **metadata.name** that you specified in the CRD configuration, for example, **sample-git**.
4. Verify that the pod status is **Running**.
5. Click the **feast** pod and then select **Pod details**.
6. Scroll down to see the online container. This container is the deployment for the online server. It makes the feature server REST API available in the OpenShift cluster.
7. Scroll up and then click **Terminal**.
8. Run the following command to verify that the **feast** CLI is installed correctly:

```
$ feast --help
```

9. To view the files for the Feature Store project, enter the following command:

```
$ ls -la
```

You should see output similar to the following:

```
.
..
data
example_repo.py
feature_store.yaml
__init__.py
__pycache__
test_workflow.py
```

10. To view the **feature_store.yaml** configuration file, enter the following command:

```
$ cat feature_store.yaml
```

You should see output similar to the following:

```
project: my_feast_project
provider: local
online_store:
 path: /feast-data/online_store.db
 type: sqlite
registry:
 path: /feast-data/registry.db
 registry_type: file
auth:
 type: no_auth
entity_key_serialization_version: 3
```

The **feature_store.yaml** file defines the following components:

- **project** – The namespace for the Feature Store instance. Note that this project refers to the feature project rather than the OpenShift AI data science project.

- **provider** – The environment in which Feature Store deploys and operates.
- **registry** – The location of the feature registry.
- **online_store** – The location of the online store.
- **auth** - The type of authentication and authorization (**no_auth**, **kubernetes**, or **oidc**)
- **entity_key_serialization_version** - Specifies the serialization scheme that Feature Store uses when writing data to the online store.

NOTE: Although the **offline_store** location is not included in the **feature_store.yaml** file, the Feature Store instance uses a DASK file-based offline store. In the **feature_store.yaml** file, the registry type is **file** but it uses a simple SQLite database.

Next steps

- Optionally, you can customize the default configurations for the offline store, online store, or registry by editing the YAML configuration for the Feature Store CR, as described in *Customizing your Feature Store configuration* .
- Give your ML engineers and data scientists access to the data science project so that they can create a workbench. and provide them with a copy of the **feature_store.yaml** file so that they can add it to their workbench IDE, such as Jupyter.

2.1.4. Adding feature definitions and initializing your Feature Store instance

Initialize the Feature Store instance to start using it.

When you initialize the Feature Store instance, Feature Store completes the following tasks:

- Scans the Python files in your feature repository and finds all Feature Store object definitions, such as feature views, entities, and data sources.
  Note: Feature Store reads all Python files recursively, including subdirectories, even if they do not contain feature definitions. For information on identifying Python files, such as imperative scripts that you want Feature Store to ignore, see *Specifying files to ignore* .
- Validates your feature definitions, for example, by checking for uniqueness of features within a feature view.
- Syncs the metadata about objects to the feature registry. If a registry does not exist, Feature Store creates one. The default registry is a simple Protobuf binary file on disk (locally or in an object store).
- Creates or updates all necessary Feature Store infrastructure. The exact infrastructure that Feature Store creates depends on the provider configuration that you have set in **feature_store.yaml**. For example, when you specify **local** as your provider, Feature Store creates the infrastructure on the local cluster.
  Note: When you use a cloud provider, such as Google Cloud Platform or Amazon Web Service, the **feast apply** command creates cloud infrastructure that might incur costs for your organization.

Prerequisites

- An ML engineer on your team has given you a Python file that defines features. For more information about how to define features, see *Defining features*.
- If you want to store the feature registry in cloud storage or in a database, you have configured storage for the feature registry. For example, if the provider is GCP, you have created a Cloud Storage bucket for the feature registry.
- You have the **cluster-admin** role in OpenShift.
- You have created a Feature Store instance in your data science project.

Procedure

1. In the OpenShift console, select **Workloads → Pods**.
2. Make sure that your data science project is the current project.
3. Click the **feast** pod and then select **Pod details**.
4. Scroll down to see the online container. This container is the deployment for the online server, and it makes the feature server REST API available in the OpenShift cluster.
5. Scroll up and then click **Terminal**.
6. Copy the feature definition (**.py**) file to your Feature Store directory.
7. To create a feature registry and add the feature definitions to the registry, run the following command:

```
feast apply
```

Verification

- You should see output similar to the following that indicates that the features in the feature definition file were successfully added to the registry:

```
Created project credit_scoring_local
Created entity zipcode
Created entity dob_ssn
Created feature view zipcode_features
Created feature view credit_history
Created on demand feature view total_debt_calc

Created sqlite table credit_scoring_local_credit_history
Created sqlite table credit_scoring_local_zipcode_features
```

- In the OpenShift console, select **Workloads → Deployments** to view the deployment pod.

2.1.4.1. Specifying files to ignore

When you run the **feast apply** command, Feature Store reads all Python files recursively, including Python files in subdirectories, even if the Python files do not contain feature definitions.

If you have Python files, such as imperative scripts, in your registry folder that you want Feature Store to ignore when you run the feast apply command, you should create a **.feastignore** file and add a list of paths to all files that you want Feature Store to ignore.

Example .feastignore file

```
# Ignore virtual environment
venv

# Ignore a specific Python file
scripts/foo.py

# Ignore all Python files directly under scripts directory
scripts/*.py

# Ignore all "foo.py" anywhere under scripts directory
scripts/**/foo.py
```

2.1.5. Viewing Feature Store objects in the web-based UI

You can use the Feature Store Web UI to view all registered features, data sources, entities, and feature services.

Prerequisites

- You can access the OpenShift console.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- You have enabled the Feature Store component, as described in *Enabling the Feature Store component*.
- You have created a Feature Store CRD, as described in *Creating a Feature Store instance in a data science project*.

Procedure

1. In the OpenShift console, select **Administration → CustomResourceDefinitions**.
2. To filter the list, in the **Search by Name** field, enter **feature**.
3. Click the **FeatureStore** CRD and then click **Instances**.
4. Click the name of the instance that corresponds to the metadata name you specified when you created the Feature Store instance.
5. Edit the YAML to include a reference to **services.ui** in the **spec** section, as shown in the following example:

```
spec:
```

```
feastProject: credit_scoring_local
feastProjectDir:
  git:
    ref: 598a270
    url: 'https://github.com/feast-dev/feast-credit-score-local-tutorial'
services:
  ui: {}
```

6. Click **Save** and then click **Reload**.
   The Feature Store Operator starts a container for the web-based Feature Store UI and creates an OpenShift route that provides the URL so that you can access it.

7. In the OpenShift console, select **Workloads → Pods**.

8. Make sure that your project (for example, **my-ds-project**) is selected.
   You should see a deployment for the web-based UI. Note that OpenShift enables TLS by default at runtime.

9. To populate the web-based UI with the objects in your Feature Store instance:

   a. In the OpenShift console, select **Workloads → Pods**.

   b. Make sure that your project (for example, **my-ds-project**) is selected.

   c. Click the **feast** pod and then select **Pod details**.

   d. Click **Terminal**.

   e. To update the Feature Store instance, enter the following command:

   ```
   feast apply
   ```

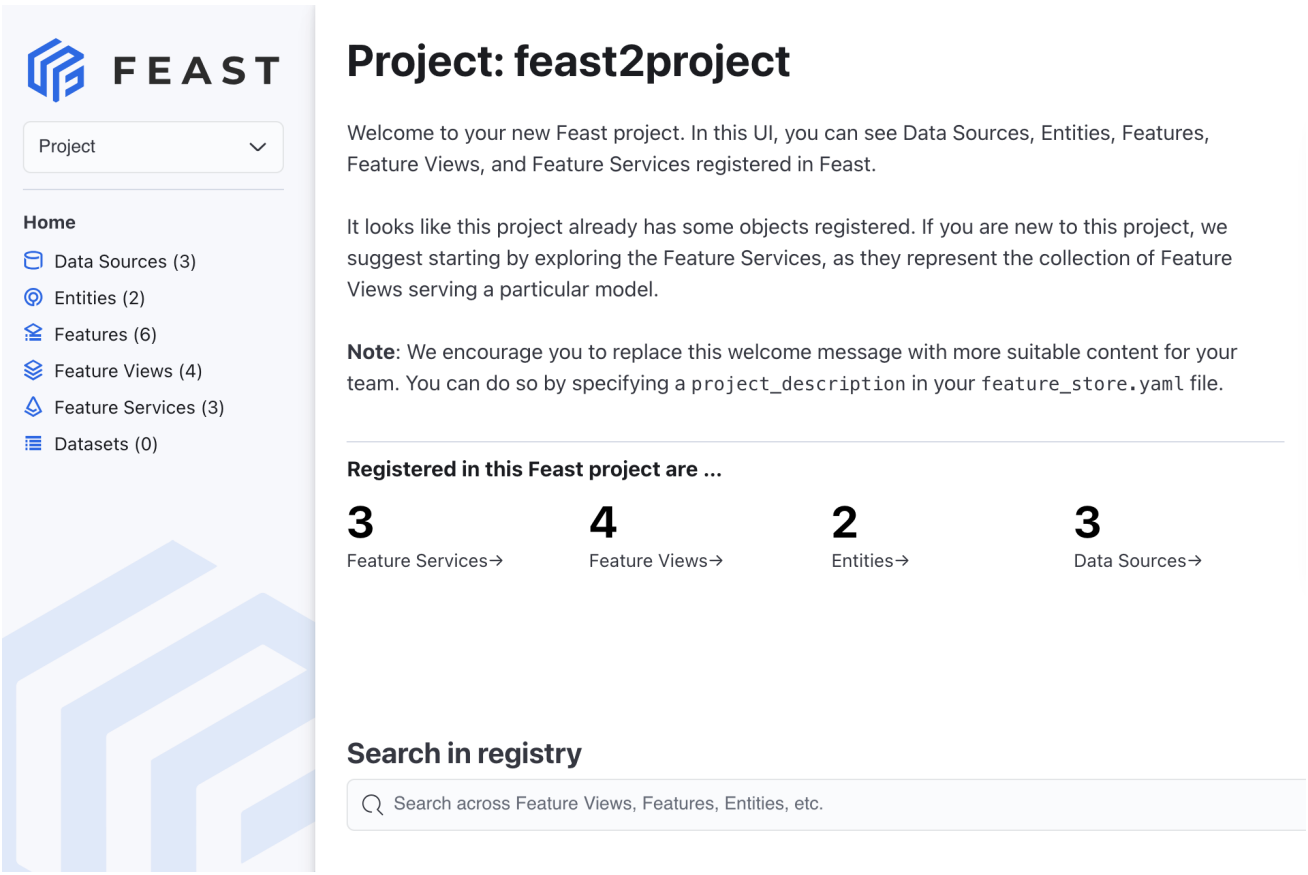
10. To find the URL for the Feature Store UI, in the OpenShift console, click **Networking → Routes**. In the row for the Feature Store UI, for example **feast-sample-ui**, the URL is in the **Location** column.

11. Click the URL link to open it in your default web browser.

Verification

The Feature Store Web UI is displayed and shows the feature objects in your project as shown in the following figure:

Figure 2.1. The Feature Store Web UI



2.2. CUSTOMIZING YOUR FEATURE STORE CONFIGURATION

Optionally, you can apply the following configurations to your Feature Store instance:

- Configure an offline store
- Configure an online store
- Configure the feature registry
- Configure persistent volume claims (PVCs)
- Configure role-based access control (RBAC)

The examples in the following sections describe how to customize a feature store instance by creating a new custom resource definition (CRD). Alternatively, you can customize an existing feature instance as described in *Editing an existing feature store instance* .

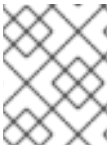
For more information about how you can customize your feature store configuration, see the Feast API documentation.

2.2.1. Configuring an offline store

When you create a Feature Store instance that uses the minimal configuration, by default, Feature Store uses a SQLite file-based store for the offline store.

The example in the following procedure shows how to configure DuckDB for the offline store.

You can configure other offline stores, such as Snowflake, BigQuery, Redshift, as detailed in the Feast reference documentation for offline stores.



NOTE

The example code in the following procedure requires that you edit it with values that are specific to your use case.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have enabled the Feature Store component, as described in *Enabling the Feature Store component*.
- You have created a data science project, as described in Creating a data science project . In the following procedure, **my-ds-project** is the name of the data science project.
- Your data science project includes an existing secret that provides credentials for accessing the database that you want to use for the offline store. The example in the following procedure requires that you have configured DuckDB.

Procedure

1. In the OpenShift console, click the **Quick Create** () icon and then click the **Import YAML** option.
2. Verify that your data science project is the selected project.
3. Copy the following code and paste it into the YAML editor:

```
apiVersion: feast.dev/v1alpha1
kind: FeatureStore
metadata:
  name: sample-db-persistence
spec:
  feastProject: my_project
  services:
    offlineStore:
      persistence:
        file:
          type: duckdb
```

4. Edit the **services.offlineStore** section to specify values specific to your use case.
5. Click **Create**.

Verification

1. In the OpenShift console, select **Workloads → Pods**.
2. Make sure that your project (for example, **my-ds-project**) is selected.

3. Find the pod that has the **feast-** prefix, followed by the metadata name that you specified in the CRD configuration, for example, **feast-sample-db-persistence**.

4. Verify that the status is **Running**.

2.2.2. Configuring an online store

When you create a Feature Store instance using the minimal configuration, by default, the online store is a SQLite database.

The example in the following procedure shows how to configure a PostgreSQL database for the online store.

You can configure other online stores, such as Snowflake, Redis, and DynamoDB, as detailed in the Feast reference documentation for online stores .

NOTE

The example code in the following procedure requires that you edit it with values that are specific to your use case.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have enabled the Feature Store component, as described in *Enabling the Feature Store component*.
- You have created a data science project, as described in Creating a data science project . In the following procedure, **my-ds-project** is the name of the data science project.
- Your data science project includes an existing secret that provides credentials for accessing the database that you want to use for the online store. The example in the following procedure requires that you have configured a PostgreSQL database.

Procedure

1. In the OpenShift console, click the **Quick Create** () icon and then click the **Import YAML** option.

2. Verify that your data science project is the selected project.

3. Copy the following code and paste it into the YAML editor:

```
apiVersion: feast.dev/v1alpha1
kind: FeatureStore
metadata:
  name: sample-db-persistence
spec:
  feastProject: my_project
  services:
    onlineStore:
      persistence:
        store:
```

```
    type: postgres
    secretRef:
      name: feast-data-stores
```

4. Edit the **services.onlineStore** section to specify values that are specific to your use case.
5. Click **Create**.

Verification

1. In the OpenShift console, select **Workloads → Pods**.
2. Make sure that your project (for example, **my-ds-project**) is selected.
3. Find the pod that has the **feast-** prefix, followed by the metadata name that you specified in the CRD configuration, for example, **feast-sample-db-persistence**.
4. Verify that the status is **Running**.

2.2.3. Configuring the feature registry

By default, when you create a feature instance using the minimal configuration, the registry is a simple SQLite database.

The example in the following procedure shows how to configure an S3 registry.

You can configure other types of registries, such as GCS, SQL, Snowflake, as detailed in the Feast reference documentation for registries.

NOTE

The example code in the following procedure requires that you edit it with values that are specific to your use case.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have enabled the Feature Store component, as described in *Enabling the Feature Store component*.
- You have created a data science project, as described in Creating a data science project . In the following procedure, **my-ds-project** is the name of the data science project.
- Your data science project includes an existing secret that provides credentials for accessing the database that you want to use for the registry. The example in the following procedure requires that you have configured S3.

Procedure

1. In the OpenShift console, click the **Quick Create** () icon and then click the **Import YAML** option.
2. Verify that your data science project is the selected project.

3. Copy the following code and paste it into the YAML editor:

```
apiVersion: feast.dev/v1alpha1
kind: FeatureStore
metadata:
 name: sample-s3-registry
spec:
 feastProject: my_project
 services:
   registry:
     local:
       persistence:
         file:
           path: s3://bucket/registry.db
           s3_additional_kwargs:
             ServerSideEncryption: AES256
             ACL: bucket-owner-full-control
             CacheControl: max-age=3600
```

4. Edit the **services.registry** section to specify values that are specific to your use case.

5. Click **Create**.

Verification

1. In the OpenShift console, select **Workloads → Pods**.

2. Make sure that your project (for example, **my-ds-project**) is selected.

3. Find the pod that has the **feast-** prefix, followed by the metadata name that you specified in the CRD configuration, for example, **sample-s3-registry**.

4. Click the feast pod and then select **Pod details**.

5. Click **Terminal**.

6. In the Terminal window, enter the following command to view the configuration, including the S3 registry:

```
$ cat feature_store.yaml
```

2.2.4. Example PVC configuration

When you configure the online store, offline store, or registry, you can also configure persistent volume claims (PVCs) as shown in the following Feature Store custom resource definition (CRD) example.

NOTE

The following example code requires that you edit it with values that are specific to your use case.

```
apiVersion: feast.dev/v1alpha1
kind: FeatureStore
metadata:
```

```
  name: sample-pvc-persistence
spec:
  feastProject: my_project
  services:
    onlineStore:  (1)
      persistence:
        file:
          path: online_store.db
          pvc:
            ref:
              name: online-pvc
            mountPath: /data/online
    offlineStore:  (2)
      persistence:
        file:
          type: duckdb
          pvc:
            create:
              storageClassName: standard
              resources:
                requests:
                  storage: 5Gi
            mountPath: /data/offline
    registry:  (3)
      local:
        persistence:
          file:
            path: registry.db
            pvc:
              create: {}
              mountPath: /data/registry
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: online-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

(1) The online store specifies a PVC that must already exist.

(2) The offline store specifies a storage class name and storage size.

(3) The registry configuration specifies that the Feature Store Operator creates a PVC with default settings.

2.2.5. Editing an existing Feature Store instance

The examples in this document describe how to customize a Feature Store instance by creating a new custom resource definition (CRD). Alternatively, you can customize an existing feature instance.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have created a Feature Store instance, as described in *Deploying a Feature Store instance in a data science project*.

Procedure

1. In the OpenShift console, select **Administration → CustomResourceDefinitions**.
2. To filter the list, in the **Search by Name** field, enter **feature**.
3. Click the **FeatureStore** CRD and then click **Instances**.
4. Select the instance that you want to edit, and then click **YAML**.
5. In the YAML editor, edit the configuration.
6. Click **Save** and then click **Reload**.

Verification

The Feature Store instance CRD deploys successfully.

CHAPTER 3. DEFINING MACHINE LEARNING FEATURES

As part of the Feature Store workflow, machine learning (ML) engineers or data scientists are responsible for identifying data sources and defining features of interest.

3.1. SETTING UP YOUR WORKING ENVIRONMENT

You must set up your Red Hat OpenShift AI working environment so that you can use features in your machine learning workflow.

Prerequisites

- You have access to the OpenShift AI data science project in which your cluster administrator has set up the Feature Store instance.

Procedure

1. From the OpenShift AI dashboard, click **Data science projects**.
2. Click the name of the project in which your cluster administrator has set up the Feature Store instance.
3. In the data science project in which the cluster administrator set up Feature Store, create a workbench, as described in Creating a workbench .
4. To open the IDE (for example, JuypterLab), in a new window, click the open icon () next to the workbench.
5. Add a **feature_store.yaml** file to your notebook environment. For example, upload a local file or clone a Git repo that contains the file, as described in Uploading an existing notebook file to JupyterLab from a Git repository by using the CLI.
6. Open a new Python notebook.
7. In a cell, run the following command to install the **feast** CLI:

```
! pip install feast
```

Verification

1. Run the following command to list the available features:

```
! feast features list
```

The output should show a list of features, Feature View and data type similar to the following:

```
Feature Feature        View          Data Type
credit_card_due        credit_history    Int64
mortgage_due           credit_history    Int64
student_loan_due       credit_history    Int64
vehicle_loan_due       credit_history    Int64
```

```
city           zipcode_features  String
state          zipcode_features  String
location_type      zipcode_features  String
```

2. Optionally, run the following commands to list the registered feast projects, feature views, and entities.

```
! feast projects list

! feast feature-views list

! feast entities list
```

3.2. ABOUT FEATURE DEFINITIONS

A machine learning feature is a measurable property or *field* within a data set that a machine learning model can analyze to learn patterns and make decisions. In Feature Store, you define a feature by defining the name and data type of a field.

A feature definition is a schema that includes the field name and data type, as shown in the following example:

```
from feast import Field
from feast.types import Int64

credit_card_amount_due = Field(
    name="credit_card_amount_due",
    dtype=Int64
)
```

For a list of supported data types for fields in Feature Store, see the **feast.types** module in the Feast documentation.

In addition to field name and data type, a feature definition can include additional metadata, specified as descriptions of features, as shown in the following example:

```
from feast import Field
from feast.types import Int64


credit_card_amount_due = Field(
    name="credit_card_amount_due",
    dtype=Int64,
    description="Credit card amount due for user",
    tags={"team": "loan_department"},
)
```

3.3. SPECIFYING THE DATA SOURCE FOR FEATURES

As an ML engineer or a data scientist, you must specify the data source for the features that you want to define.

The data source differs depending on whether you are using an offline store, for batch data and training

data sets, or an online store, for model inference. Optionally, you can use a Parquet or a Delta-formatted file as the data source. You can specify a local file or a file in storage, such as Amazon Simple Storage Service (S3).

For offline stores, specify a batch data source. You can specify a data warehouse, such as BigQuery, Snowflake, Redshift, or a data lake, such as Amazon S3 or Google Cloud Platform (GCP). You can use Feature Store to ingest and query data across both types of data sources.

For online stores, specify a database backend, such as Redis, GCP Datastore, or DynamoDB.

Prerequisites

- You know the location of the data source for your ML workflow.

Procedure

1. In the editor of your choice, create a new Python file.
2. At the beginning of the file, specify the data source for the features that you want to define within the file.
   For example, use the following code to specify the data source as a Parquet-formatted file:

```
from feast import FileSource
from feast.data_format import ParquetFormat

parquet_file_source = FileSource(
    file_format=ParquetFormat(),
    path="file:///feast/customer.parquet",
)
```

3. Save the file.

3.4. ABOUT ORGANIZING FEATURES BY USING ENTITIES

Within a feature view, you can group features that share a conceptual link or relationship together to define an entity. You can think of an entity as a primary key that you can use to fetch features. Typically, an entity maps to the domain of your use case. For example, a fraud detection use case could have customers and transactions as their entities, with group-related features that correspond to these customers and transactions.

A feature does not have to be associated with an entity. For example, a feature of a customer entity could be the number of transactions they have made on an average month, while a feature that is not observed on a specific entity could be the total number of transactions made by all users in the last month.

```
customer = Entity(name='dob_ssn', join_keys=['dob_ssn'])
```

The entity name uniquely identifies the entity. The join key identifies the physical primary key on which feature values are joined together for feature retrieval.

The following table shows example data with a single entity column (**dob_ssn**) and two feature columns (**credit_card_due** and **bankruptcies**).

Table 3.1. Example data showing an entity and features

row	timestamp	dob_ssn	credit_card_due	bankruptcies
1	5/22/2025 0:00:00	19530219_5179	833	0
2	5/22/2025 0:00:00	19500806_6783	1297	0
3	5/22/2025 0:00:00	19690214_3370	3912	1
4	5/22/2025 0:00:00	19570513_7405	8840	0

3.5. CREATING FEATURE VIEWS

You define features within a feature view. A *feature view* is an object that represents a logical group of time-series feature data in a data source. Feature views indicate to Feature Store where to find your feature values, for example, in a parquet file or a BigQuery table.

By using feature views, you define the existing feature data in a consistent way for both an offline environment, when you train your models, and an online environment, when you want to serve features to models in production.

Feature Store uses feature views during the following tasks:

- Generating training datasets by querying the data source of feature views to find historical feature values. A single training data set can consist of features from multiple feature views.
- Loading feature values into an online or offline store. Feature views determine the storage schema in the online or offline store. Feature values can be loaded from batch sources or from stream sources.
- Retrieving features from the online or offline store. Feature views provide the schema definition for looking up features from the online or offline store.

When you create a feature project, the **feature_repo** subfolder includes a Python file that includes example feature definitions (for example, **example_features.py**) .

To define new features, you can edit the code in the example file or add a new file to the feature repository.

Note: Feature views only work with timestamped data. If your data does not contain timestamps, insert dummy timestamps. The following example shows how to create a table with dummy timestamps for PostgreSQL-based data:

```
CREATE TABLE employee_metadata (
  employee_id INT PRIMARY KEY,
  department TEXT,
  dummy_event_timestamp TIMESTAMP DEFAULT '2024-01-01'
);
INSERT INTO employee_metadata (employee_id, department)
VALUES (1, 'Advanced'), (2, 'New');
```

Prerequisites

- You know what data is relevant to your use case.
- You have identified attributes in your data that you want to use as features in your ML models.

Procedure

1. In your IDE, such as JupyterLab, open the **feature_repo/example_features.py** file that contains example feature definitions or create a new Python (**.py**) file in the **feature_repo** directory.
2. Create a feature view that is relevant to your use case based on the structure shown in the following example:

```
credit_history_source = FileSource(  (1)
 name="Credit history",
 path="data/credit_history.parquet",
 file_format=ParquetFormat(),
 timestamp_field="event_timestamp",
 created_timestamp_column="created_timestamp",
)
credit_history = FeatureView(  (2)
 name="credit_history",
 entities=[dob_ssn],  (3)
 ttl=timedelta(days=90),  (4)
 schema=[  (5)
    Field(name="credit_card_due", dtype=Int64),
    Field(name="mortgage_due", dtype=Int64),
    Field(name="student_loan_due", dtype=Int64),
    Field(name="vehicle_loan_due", dtype=Int64),
    Field(name="hard_pulls", dtype=Int64),
    Field(name="missed_payments_2y", dtype=Int64),
    Field(name="missed_payments_1y", dtype=Int64),
    Field(name="missed_payments_6m", dtype=Int64),
    Field(name="bankruptcies", dtype=Int64),
 ],
 source=credit_history_source,  (6)
   tags={"origin": "internet"},  (7)
)
```

(1) A data source that provides time-stamped tabular data. A feature view must always have a data source for the generation of training datasets and when materializing feature values into the online store. Possible data sources are batch data sources from data warehouses (BigQuery, Snowflake, Redshift), data lakes (S3, GCS), or stream sources. Users can push features from data sources into Feature Store, and make the features available for training or batch scoring ("offline"), for realtime feature serving ("online"), or both.

(2) A name that identifies the feature view in the project. Within a feature view, feature names must be unique.

(3) Zero or more entities. Feature views generally contain features that are properties of a specific object, in which case that object is defined as an entity and included in the feature view. If the features are not related to a specific object, the feature view might not have entities.

(4)

(Optional) Time-to-live (TTL) to limit how far back to look when Feature Store generates historical datasets.

5 One or more feature definitions.

6 A reference to the data source.

7 (Optional) You can add metadata, such as tags that enable filtering of features when viewing them in the UI, listing them by using a CLI command, or by querying the registry directly.

3. Save the file.

CHAPTER 4. RETRIEVING FEATURES FOR MODEL TRAINING

After a cluster administrator configures Feature Store on Red Hat OpenShift AI, an ML engineer or a data scientist can retrieve features to train models for inference.

4.1. MAKING FEATURES AVAILABLE FOR REAL-TIME INFERENCE

To make features available for real-time inference for use by ML engineers and data scientists on your team, load or *materialize* feature data to the online store.

Materialization ensures that the same features are available for both model training and real-time predictions, making your ML workflow robust and reproducible. The online store serves the latest features to models for online prediction. Data scientists on your team with access to your data science project can access the features in the online store.

Note: Feature Store uses a "push" method for online serving. This means that the Feature Store pushes feature values to the online store, which reduces the latency of feature retrieval. This is more efficient than a "pull" method, where the model serving system would make a request to the Feature Store to retrieve feature values.

Prerequisites

- In your IDE environment, you have installed the Feature Store Python SDK as described in *Setting up your working environment*.
- Your cluster administrator has pushed feature data to the online store.
- Optionally, you have cloned a Git repository that includes your model training code.

Procedure

1. To load feature data to the online store, run the **feast materialize** command and specify an historical time range (start and end) for the feature data that you want to load, as shown in the following example:

```
$ feast materialize 2020-01-01T00:00:00 2022-01-01T00:00:00
```

   Feature Store queries the batch sources for all feature views over the provided time range, and loads the latest feature values into the configured online store, as indicated by output similar to the following:

```
Materializing 2 feature views to 2025-07-23 17:02:13+00:00 into the sqlite online store.

zipcode_features from 2015-07-26 17:02:18+00:00 to 2025-07-23 17:02:13+00:00:
credit_history from 2025-04-24 17:02:22+00:00 to 2025-07-23 17:02:13+00:00:
```

Verification

Run the following **feast** commands to verify that the feature data is in the online store:

```
$ feast feature-views list
```

Example output:

```
NAME          ENTITIES  TYPE
zipcode_features  {'zipcode'}  FeatureView
credit_history {'dob_ssn'}  FeatureView
total_debt_calc   {'dob_ssn'}  OnDemandFeatureView
```

```
$ feast entities list
```

Example output:

```
Output:
NAME  DESCRIPTION    TYPE
zipcode              ValueType.INT64
dob_ssn  Date of birth and last four digits of social security number  ValueType.STRING
```

4.2. RETRIEVING ONLINE FEATURES FOR MODEL INFERENCE

After your cluster administrator loads feature data to the online store, you can retrieve features for model training.

When the cluster administrator creates a **FeatureStore** CR, by default, it runs the online store feature server. This Python feature server is an HTTP endpoint that serves features with JSON I/O. You can write and read features from the online store by using any programming language that can make HTTP requests.

Note: You can also retrieve online features by using the Feast SDK as described in the upstream Feast documentation.

A feature service is an object that represents a logical group of features from one or more feature views. With a feature service, your ML models can access features from within a feature view as needed.

Typically, you create one feature service per model version, allowing for tracking of the features used by models. The features retrieved from the online store can belong to many feature views.

Note: When you apply a feature service, an actual Feature Store server is not deployed.

Prerequisites

- In your IDE environment, you have installed the Feature Store Python SDK and added a **feature_store.yaml** file, as described in *Setting up your working environment*.
- Optionally, you have cloned a Git repository that includes your model training code.
- Feature data is loaded in the online store, as described in *Making features available for real-time inference*.

Procedure

1. From the OpenShift AI dashboard, click **Data science projects**.
2. Click the name of the project and then click the **Workbenches** tab.
3. Click the open icon ( ) next to the workbench.

4. In a Python notebook cell, run the following command to instantiate a local FeatureStore object that can parse the feature registry:

```
from feast import FeatureStore

feature_store = FeatureStore('.')
```

5. Identify the feature service:

```
feature_service = feature_store.get_feature_service("credit_activity")
```

6. Retrieve features from the online store by using the **get_online_features** method:

```
features = feature_store.get_online_features(
features=feature_service,
    entity_rows=[
        # {join_key: entity_value}
        {"dob_ssn": 19530219_5179},
        {"dob_ssn": 19500806_6783},
    ]
)
```

Verification

When you run the **feast** commands, you should see output similar to the following:

Example output

```
{
 'credit_card_due': [833,1297],
 'bankruptcies': [0, 1],
 'dob_ssn': [19530219_5179, 19500806_6783]
}
```

Next step

- See the following Git repositories for an example of how to use features to train a model.
  https://github.com/feast-dev/feast-credit-score-local-tutorial

  https://github.com/feast-dev/feast/tree/stable/examples/operator-quickstart